



HABIB UNIVERSITY

Data Structures & Algorithms

CS/CE 102/171 Spring 2025

Instructor: Maria Samad

Date: February 24th, 2025

Quiz # 2

Time Allowed: 45 minutes

Max Points: 30

Name: _____

QUESTION 1: ALGORITHM DESIGN USING CIRCULAR QUEUES (10 marks)

- **Log Management in a Circular Queue:**
 - The user inputs an expiration time in minutes, which should be an integer of not more than 59. If the expiration time is not in the valid range, continue taking input until right value is received.
 - Design an algorithm to design and manage a circular queue to manage system log entries, as explained below.
 - Circular Queue is of size 'n', which is initially empty. Assume, both circular queue and n are correct, so you may use them without error checking and taking inputs. However, you must declare/define them in your algorithm.
 - Each log entry is defined as a tuple consisting of the timestamp at which it occurs, and a string message, which is the log message. See example below to know the exact format.
 - The log entries are going to expire after the input expiration time in the first step is over, meaning that any log older than the expiration time should automatically be removed from the circular queue.
 - This means that before enqueueing log entries, the previous log entries must be checked, and if the time difference between their timestamps and current time is beyond the expiration time, it should be dequeued.
 - Once the expired log entries have been removed, only then the current log entry should be added to the circular queue, according to the behavior of circular queues.
 - While adding, if the queue is already full, it should first remove the oldest log before adding a new one.
 - For example:
 - Circular Queue size is 5, and expiration time is 10 minutes
 - Initial Circular Queue of size 5 = [None, None, None, None, None]
 - Front pointer = Rear pointer = None
 - Summary of Operations, as the log stamp appears is as follows:
 1. Enqueue (1, "Log 1"): Queue = [(1, "Log 1"), None, None, None, None]
 2. Enqueue (3, "Log 2"): Queue = [(1, "Log 1"), (3, "Log 2"), None, None, None]
 3. Enqueue (5, "Log 3"): Queue = [(1, "Log 1"), (3, "Log 2"), (5, "Log 3"), None, None]
 4. Enqueue (15, "Log 4") (remove expired logs): Queue = [(5, "Log 3"), (15, "Log 4"), None, None, None]
 5. Enqueue (20, "Log 5") (remove expired logs): Queue = [(15, "Log 4"), (20, "Log 5"), None, None, None]
 6. Enqueue (21, "Log 6"): Queue = [(15, "Log 4"), (20, "Log 5"), (21, "Log 6"), None, None]
 7. Enqueue (22, "Log 7"): Queue = [(15, "Log 4"), (20, "Log 5"), (21, "Log 6"), (22, "Log 7"), None]
 8. Enqueue (23, "Log 8"): Queue = [(15, "Log 4"), (20, "Log 5"), (21, "Log 6"), (22, "Log 7"), (23, "Log 8")]
 9. Enqueue (24, "Log 9") (queue is full so remove the front element, which in this situation at the moment is "Log 4" timestamp, to make new room and then add (24, "Log 9") to CQ as following: Queue = [(20, "Log 5"), (21, "Log 6"), (22, "Log 7"), (23, "Log 8"), (24, "Log 9")]
 10. Enqueue (-1, "EXIT"): Indication to stop the program
- **REQUIREMENTS:**
 - This is a question on CIRCULAR QUEUES, so please stick to ONLY Circular Queues for the algorithm. The enqueue and dequeue functions do NOT handle the CQ pointers, so you should handle them in your algorithm.
 - Do not **HARD CODE** your design for the given example. The algorithm should work for any valid sized CQs.
 - Declare all the data structures and variables being used in this algorithm, including any inputs, if any.
 - The program stops when a log entry with timestamp -1 and string message, "EXIT" occurs
 - The algorithm should be able to handle new log additions and automatically discard expired/old entries efficiently, i.e. in constant time, as per the behavior of circular queues
 - **DO NOT USE ANY OTHER DATA STRUCTURES INCLUDING ListADT/Python lists apart from the ones permitted to use**
 - **REMEMBER:** Circular Queues are non-iterable and non-traversing data structures, so you CANNOT and MUST NOT do indexing within the data structure to access the elements.

QUESTION 2: BINARY SEARCH OPERATIONS (10 marks)

A sorted list of student IDs is given as follows: **studIDs = [3, 7, 12, 18, 24, 32, 39, 45, 51, 62, 71]**

Answer the following questions based on the binary search process, by showing all the steps and/or calculations. Only giving final answers will **NOT** be given any credit.

1. List the sequence of indices that the binary search algorithm will examine while searching for student ID = 32.
2. Suppose the student mistakenly searches for roll number 60 instead. List the sequence of indices examined and explain how binary search determines that 60 is not in the array.
3. List the sequence of indices that the binary search algorithm will examine while searching for student ID = 3.
4. If a new roll number 34 is added to the list while maintaining sorted order, what is the effect on the binary search process when searching for 32, and then 39? Which indices would now be visited?
5. If 18 is deleted, how does the search path change for 24 and 12?
6. What is the maximum number of comparisons binary search makes in a sorted array of size 1000?
7. Given a sorted array of 19 elements, what will be the first middle index chosen by binary search? What if the array had 20 elements instead?

QUESTION 3: BUBBLE SORT (10 marks)

Sort the given list in descending order using bubble sort algorithm.

- [43, 92, 31, 14, 27, 65, 79, 16, 58, 90, 2]

SHOW ALL INTERIM CALCULATIONS IN ORDER TO GET COMPLETE/FULL MARKS. An answer without the intermediate steps will directly result in 0 marks.

